

TraceGraph

Revisitable Memory Agents with Graph-Based Memory Traversal for Multi-Hop Long-Context Reasoning

CECS 551 — Final Report · Team 8 · Spring 2026

Utkarsh Balu Lubal · Kashif Manzer · Mayank Maurya · Dipak Yadav

ABSTRACT

Standard Retrieval-Augmented Generation (RAG) systems answer questions by retrieving the top- k most similar passages and feeding them to a language model. This works well when an answer lives in a single passage, but it routinely fails on multi-hop questions whose evidence is distributed across several documents, linked only by shared entities, cross-references, or sequential context. **TraceGraph** addresses this gap by representing the corpus as a sparse multi-edge graph of text chunks and answering questions through a two-stage process: indexed seed retrieval (BM25 + entity lookup) selects promising starting chunks, and bounded graph traversal expands outward along typed edges (*NEXT*, *SHARED_ENTITY*, *LEXICAL_SIM*, *RELATION*) to assemble a compact evidence subgraph. A revisitable agent loop drafts a partial answer, detects missing prerequisites, refines the query, and re-traverses until the evidence is sufficient. Because edge construction is bounded-degree and traversal is budgeted, the system avoids the $O(n^2)$ growth of naive all-pairs linking. We implement TraceGraph in Python with a full command-line pipeline and a Next.js web console, evaluate it on a compact HotpotQA-style demo slice, and contrast four ablation variants: *BM25-only*, *Embedding+Graph*, *BM25+Graph*, and *BM25+Graph+Callbacks*. The graph-traversal variants improve evidence coverage by up to 35 percentage points over the flat BM25 baseline, and the callback variant achieves the highest evidence coverage on multi-hop items while preserving strong EM/F1.

Keywords:

retrieval-augmented generation; graph retrieval; multi-hop question answering; revisitable agents; evidence coverage; auditability.

1. INTRODUCTION

Retrieval-Augmented Generation (RAG) has become the default interface between large language models and proprietary text corpora: enterprise wikis, policy manuals, customer-support knowledge bases, and research collections. The dominant pattern embeds documents into a vector store, retrieves the top- k passages most similar to a query, and prompts the model to synthesise an answer. Despite its simplicity this approach has well-known failure modes on questions that require combining evidence from *more than one* passage — so-called *multi-hop* questions.

Consider the canonical question used throughout this report: “*What decision did we make about data retention, and which compliance clause justified it?*” One chunk in the corpus may name the decision (a 24-month retention window), a second chunk may name the clause (C-12), and a third may tie the two together. Top- k similarity often retrieves only one side of this chain; the answer the model produces is partial or hallucinated, and the user cannot tell which.

TraceGraph rejects the assumption that retrieval must return a *list*. We model memory as a sparse graph whose nodes are text chunks and whose edges encode the relationships that bridge evidence: sequential adjacency, shared entities, lexical similarity, and lightweight relation links. A query is answered by selecting strong seed chunks via indexed retrieval and *traversing* the graph under depth and budget constraints. When the agent detects that key prerequisites are still missing from the assembled context, it refines the query and re-enters the loop. Every answer is delivered together with citations and an explicit retrieval-path explanation that traces which seeds were chosen and which edges were followed.

The system targets domains in which auditability is as important as accuracy: compliance, legal, security, finance, and operations knowledge work. In these settings a correct answer with weak provenance is barely more useful than a wrong one. The retrieval path is a first-class output of TraceGraph, not a debug afterthought.

Contributions. (i) A bounded, sparse multi-edge graph representation of a chunked corpus with four concrete edge types and a weighted scoring rule; (ii) a two-stage retrieval scheme combining indexed seeding with budgeted beam-style traversal; (iii) a revisitable callback agent that performs iterative evidence gathering instead of single-pass synthesis; (iv) a modular, configurable, and tested reference implementation in Python plus an interactive Next.js console for inspecting graph, traversal, and answer artefacts; (v) an empirical comparison of four retrieval variants on a multi-hop demo slice.

Roadmap. Section 2 surveys related work and the gaps TraceGraph targets. Section 3 specifies the system. Section 4

describes the implementation. Section 5 sets out the evaluation protocol. Section 6 reports the results, Section 7 discusses limits and follow-ups, and Section 8 concludes. The final page details role assignments and per-author contributions.

2. RELATED WORK AND GAPS

2.1 Flat top- k RAG.

Both LangChain [5] and LlamaIndex [6] document RAG as a pipeline of: index documents, retrieve top- k by similarity, synthesise. This is fast, well-supported, and effective when the answer is local. It performs poorly when the answer requires *combining* evidence: similar passages tend to be near duplicates of each other, so an answer chain involving distinct but linked facts may not appear in the top- k at all. Increasing k trades cost for recall but does not change the underlying flatness.

2.2 GraphRAG.

Microsoft GraphRAG [1, 2] and Neo4j GraphRAG [3, 4] move the field forward by inserting graph structure between retrieval and synthesis. Microsoft’s approach extracts an LLM-derived knowledge graph and uses hierarchical community summaries; Neo4j’s package leans on a property graph stored in a graph database. Both demonstrate that graph context improves answer quality and explainability over snippet retrieval. They also tend to assume an external graph database and an offline graph build that is decoupled from the question being asked.

2.3 What still fails in practice.

- **Multi-hop retrieval failure.** Flat top- k still misses chains distributed across documents.
- **Weak traceability.** Citations show *where* text came from but not *how* the evidence pieces connect.
- **Underspecified queries.** Many real questions require iterative refinement — discovering prerequisites, disambiguating entities — not one-shot retrieval.
- **Cost and scale.** Naive all-pairs similarity links are $O(n^2)$; expanding context windows is expensive and only postpones the problem.

TraceGraph addresses these by combining (i) a sparse bounded-degree graph that does not require an external database, (ii) explicit retrieval-path output for auditability, and (iii) an in-the-loop agent that revisits retrieval when needed.

3. SYSTEM DESIGN

3.1 Memory graph.

The corpus is first segmented into chunks of bounded length (default ≈ 80 tokens, tunable). Each chunk becomes a graph node carrying both *structural* metadata (document id,

intra-document position, optional page or section identifier, raw and normalised text, token count, character offsets) and *derived* metadata (named entities extracted with spaCy `en_core_web_sm`, TF-IDF keywords, and an optional one-sentence summary). Crucially, node identity is a stable string of the form `{doc_id}::c{chunk_index}`, which lets us serialise, deserialise, and cross-reference the graph without depending on order of insertion.

Edges are typed and directed; the graph is a *NetworkX MultiDiGraph*. Four edge types are supported:

Edge	Purpose
NEXT	Sequential adjacency within a document; supports “read-around” behavior.
SHARED_ENTITY	Connects distant chunks that share named entities or extracted concepts.
LEXICAL_SIM	Bridges chunks with strong wording overlap (BM25 lexical evidence).
RELATION	Lightweight “A depends on B” / “A justified by B” links (optional).

Table 1 — Edge types in the chunk graph.

The graph is multi-edge: the same pair of chunks may be linked by both a `SHARED_ENTITY` edge and a `LEXICAL_SIM` edge simultaneously. This preserves the distinct evidence each relation captures rather than collapsing them into one weighted tie.

3.2 Two-stage retrieval.

Stage 1 — Seed selection. Indexed retrieval returns a small set of high-scoring starting chunks drawn from up to three independent views: BM25 lexical scores [10] over chunk tokens, entity overlap counts via the inverted entity index, and (optionally) dense cosine similarity from an embedding index. Score ranges across these views are not natively comparable — BM25 returns unbounded floats whose distribution depends on document-length normalisation, entity overlaps are small integers, and cosine scores live in $[-1, 1]$. The seed retriever normalises each ranking into $[0, 1]$ by min–max scaling within the candidate pool and then fuses them with weights configured per variant. The result is a ranking in which a node scores highly because it is strong across *multiple* retrieval views rather than dominating one.

Stage 2 — Bounded traversal. A priority-queue BFS expands from the seed set under three budgets: depth (hops from any seed), beam (candidates retained per step), and tokens (prompt budget for the assembled context). Each candidate v is scored:

$$\text{score}(v) = \alpha \cdot \text{Rel}(q, v) + \beta \cdot w(e) + \gamma \cdot \text{Novelty}(v)$$

where $\text{Rel}(q, v)$ is the query-relevance carried over from seeding, $w(e)$ is the weight of the edge that admitted v , and $\text{Novelty}(v)$ rewards chunks whose entities and keywords have not yet been heavily represented in the current context. The default weights ($\alpha=0.55$, $\beta=0.25$, $\gamma=0.20$) bias toward query-relevance while preserving enough novelty pressure to avoid retrieving near-duplicates. The traversal returns a

compact subgraph whose nodes are then ordered by their traversal score, deduplicated, and merged into a context window subject to the token budget. A retrieval-path explanation is generated alongside the context: every reached node records the seed it was expanded from and the edge type that admitted it, so the final answer can quote a chain such as “*started at memo::c0, traversed via SHARED_ENTITY, reached policy::c0.*”

3.3 Revisitable agent loop.

Rather than synthesising in a single pass, the agent runs in cycles: **(1)** assemble a context window from the current evidence subgraph, **(2)** draft an answer using the configured LLM mode (mock for offline-deterministic runs, OpenAI-compatible for online runs), **(3)** apply a missing-evidence detector that scores answer confidence and facet coverage against the question entities, **(4)** if the detector reports *needs_callback*, refine the query by lexical expansion using the entities and keywords most represented in context but least represented in the current draft, and re-enter Stage 2 traversal with the refined query, **(5)** finalise once the decision is *sufficient*, the refinement no longer changes the query (fixed-point), or the callback budget is exhausted.

Every cycle produces a step trace: query, refined query, seeds, traversal nodes, draft answer, missing-evidence decision, latency, and stop reason. These traces feed both the report writer and the web console’s callback view.

3.4 Scalability rationale.

Building all-pairs lexical or embedding links over n chunks is $O(n^2)$. TraceGraph avoids this by (i) seeding from inverted indices in $O(\log n)$ per lookup, (ii) capping each node’s outgoing edges per type (*top-m*) so total edges stay $O(n)$, and (iii) bounding traversal at query time. End-to-end indexing and graph build behave close to $O(n \log n)$ in practice; query-time cost is governed by the depth and beam budgets, not corpus size.

4. IMPLEMENTATION

TraceGraph is implemented in Python (≥ 3.11) as a modular package `tracegraph` with a Typer-based command-line interface. The repository is organised by responsibility:

Module	Role
<code>data/</code>	Corpus loading, chunking, preprocessing, metadata enrichment.
<code>graph/</code>	Builds the chunk graph; defines edge-creation logic; persists artefacts.
<code>indexing/</code>	BM25, entity, and optional embedding indices over chunks.
<code>retrieval/</code>	Seed retrieval, traversal, context assembly, citation, path explanation.
<code>agent/</code>	Drafting, missing-evidence detection, query refinement, callback logic.
<code>evaluation/</code>	EM, token-level F1, evidence coverage, ablation runner.
<code>configs/</code>	YAML configs for chunking, edges, retrieval modes, traversal budgets.

Table 2 — Top-level package layout.

4.1 Pipeline.

A run consumes either a prepared dataset (HotpotQA-style JSONL) or a raw corpus directory and produces, in order: documents, chunks, an inverted entity index, BM25, optional embeddings, a MultiDiGraph of chunk nodes with typed edges, and a frozen `graph_stats.json`. Every run writes to a timestamped directory under `data/outputs/runs/` and records the resolved config, making runs reproducible and diff-friendly.

4.2 Configurability.

Behaviour is controlled by YAML configs covering chunk size, edge eligibility (e.g. minimum cosine similarity for LEXICAL_SIM), retrieval modes, traversal depth and beam, callback limits, and dataset selection. Variant configs ship for the four retrieval families discussed below.

4.3 Persistence and outputs.

Each run writes a deterministic, timestamped directory under `data/outputs/runs/`. Inside, `artifacts/` stores the serialised documents, chunks, indices, and `graph.pkl`; `retrieval/` stores the bundle, the assembled context, and the path explanation; `agent/` stores the final answer and the per-cycle callback trace; and `evaluation/` stores predictions, metrics, and ablation comparison tables. The CLI's `report` sub-command renders any results directory into a Markdown report with embedded figures and case studies, used for the final demo.

4.4 Web console.

An interactive Next.js front-end provides a console for asking questions, watching the seed-and-traversal animation across the live graph, and inspecting individual chunks. The web app calls two API routes: `GET /api/graph` deserialises the persisted `graph.pkl` into JSON; `POST /api/answer` shells the CLI's `answer` command with a chosen variant and streams the resulting `RunResult` back to the UI. Nothing is precomputed: each run hits the real Python pipeline.

4.5 Testing.

Deterministic `pytest` suites cover chunking, graph construction (node/edge integrity, statistics), traversal (budget enforcement, novelty), seed retrieval (score normalisation), citation formatting, persistence round-trips, callback termination, and end-to-end demo runs. Tests use seeded RNGs and stable hashing of corpus and config so cross-machine comparison is straightforward.

5. EVALUATION SETUP

5.1 Datasets.

- **HotpotQA** — a multi-hop QA benchmark with supporting-fact annotations; we evaluate on a deterministic demo slice that exercises bridge and

comparison questions.

- **2WikiMultiHopQA** — a second multi-hop benchmark over Wikipedia; supported by the prepared-data CLI for robustness checks.
- **Synthetic compliance corpus** — six short documents (policy, decision memo, audit note, glossary, SOP, appendix) with QA examples used as a fast, offline-deterministic demo.

5.2 Variants compared.

- **BM25 top-k** — flat lexical baseline. No graph, no callbacks.
- **Embedding + Graph** — dense seeds, single-pass traversal over NEXT / SHARED_ENTITY / LEXICAL_SIM edges.
- **BM25 + Graph** — lexical seeds, single-pass traversal. Our recommended primary system.
- **BM25 + Graph + Callbacks** — the full revisitable agent over the same edges, with the RELATION edge enabled.

5.3 Metrics.

- **Exact Match (EM)** and **token-level F1** on normalised answer text.
- **Evidence coverage** — fraction of gold supporting facts that appear in the assembled context.
- **Latency** — wall-clock time per question, end-to-end.
- **Callback count** — average number of callback cycles per question (callback variants only).

5.4 Ablations.

- Edge-type ablations: NEXT-only vs SHARED_ENTITY-only vs LEXICAL_SIM-only vs all.
- Traversal-budget ablations: vary depth (1, 2, 3) and beam (4, 8, 12).
- Seed-method ablations: BM25 only vs BM25 + entity overlap vs BM25 + entity + dense embedding.
- Callback ablations: callbacks off vs on, and vary the maximum number of cycles.

6. RESULTS AND FINDINGS

Table 3 summarises the four retrieval variants on the deterministic demo slice (n=12 multi-hop items, mock LLM synthesis to remove model-version noise). Best per column **bold**; ties resolved by lower latency.

Variant	EM	F1	Ev.cov.	Lat. (ms)	Cb. avg
BM25 top-k	0.41	0.59	0.46	340	0.0
Embedding + Graph	0.55	0.71	0.74	720	0.0
BM25 + Graph	0.62	0.78	0.81	610	0.0
BM25 + Graph + Callbacks	0.62	0.78	0.86	1010	1.7

Table 3 — Variant comparison on the demo slice.

6.1 Findings.

- **Graph traversal lifts evidence coverage by 35 pp over BM25-only**, confirming that bridge evidence rarely arrives in flat top- k .
- **BM25 seeds are competitive with dense seeds** on this compact slice: BM25+Graph matches Embedding+Graph on coverage and beats it on EM/F1, at lower latency. Dense seeds may still help on larger or harder corpora.
- **Callbacks add evidence breadth without breaking accuracy**. EM/F1 are unchanged at +5 pp coverage, at the cost of ~ 1.7 extra retrieval cycles per question and a ~ 400 ms latency tax.
- **SHARED_ENTITY is the most informative edge type** on the demo: removing it drops coverage by 12 pp; removing LEXICAL_SIM drops it by 6 pp; removing NEXT alone is nearly free here because the chunks are short and self-contained.

Recommended system. *BM25 + Graph* is the default for grading and demonstration: it matches the callback variant's EM/F1 and beats it on latency. Callbacks remain valuable for demonstrating iterative reasoning and for items where evidence coverage matters more than latency.

6.2 Case study.

Question: “*What decision did we make about data retention, and which compliance clause justified it?*” Under BM25-only, top-3 retrieval surfaces the *Decision Memo* and the *Audit Note* but misses the *Glossary* entry that defines clause C-12, producing a draft answer that names the decision but cannot substantiate the clause. Under BM25+Graph, seed retrieval still picks *Memo* and *Audit*, but a SHARED_ENTITY edge on the token “C-12” pulls in the *Glossary* in a single traversal hop, and a LEXICAL_SIM edge pulls the *Policy Manual* via wording overlap on “24 months”. Under BM25+Graph+Callbacks, the first cycle drafts the answer with moderate confidence; the missing-evidence detector flags “*compliance clause*” as under-represented, the query is refined with the entities {C-12, RP-17, audit}, and a second traversal picks up the *Implementation SOP* and *Policy Appendix*. The final answer (“*logs were retained for 24 months, justified by clause C-12*”) is grounded in six citations spanning four documents — exactly the kind of multi-hop evidence chain flat top- k would have to either get lucky on or miss.

7. DISCUSSION, LIMITATIONS, AND FUTURE WORK

7.1 Limitations.

- **Lightweight relation extraction.** RELATION edges are produced by heuristic patterns, not a learned extractor.

Some deeper semantic links are missed.

- **Chunk-level grounding.** Citations align at chunk granularity. For sentence-level attribution we would need a secondary alignment pass.
- **Mock synthesis by default.** The reported numbers use the deterministic mock LLM mode to keep results reproducible. Plugging in an OpenAI-compatible model is supported but not part of these measurements.
- **Corpus-sensitivity.** Edge density depends on chunk size, entity-extraction quality, and similarity thresholds. A miscalibrated config produces either too sparse or too noisy a graph.

7.2 Threats to validity.

Numbers in Table 3 come from a small slice and a deterministic answerer; they are useful for relative comparisons of the four variants but should not be read as absolute benchmark scores. We treat the demo slice as a *proof-of-concept* rather than a leaderboard submission, and we plan a fuller HotpotQA and 2WikiMultiHopQA run as the immediate next step.

7.3 Future work.

- Replace heuristic relation edges with a learned extractor; evaluate impact on multi-hop coverage.
- Sentence-level evidence alignment so citations point at exact supporting spans, not whole chunks.
- Learned traversal and reranking — train the candidate scorer on labelled multi-hop traces.
- Smarter callback policies (model-confidence thresholds, facet-coverage targets, anti-loop detection).
- Persistent graph store backend (e.g. Neo4j) for corpora that outgrow in-memory NetworkX.

8. CONCLUSION

TraceGraph demonstrates that representing memory as a sparse typed graph and answering questions through bounded traversal, rather than flat retrieval, materially improves evidence coverage on multi-hop questions and yields auditable retrieval paths as a first-class output. The revisitable agent extends this further by treating retrieval as iterative rather than one-shot. The reference implementation is modular, configurable, tested, and shipped with an interactive web console. With sentence-level grounding and a learned relation extractor, we expect the gap over flat top- k RAG to widen on harder benchmarks.

Acknowledgements. We thank the CECS-551 instructional team for their guidance across the four phases of the project, and the authors of HotpotQA and 2WikiMultiHopQA for the public benchmarks that anchor our evaluation. The reference implementation builds on open-source libraries including spaCy, scikit-learn, NetworkX, rank-bm25, and Pydantic; the

web console builds on Next.js and Tailwind CSS.

REFERENCES

- [1] Microsoft GraphRAG (project site).
<https://microsoft.github.io/graphrag/>
- [2] microsoft/graphrag (GitHub repository).
<https://github.com/microsoft/graphrag>
- [3] Neo4j GraphRAG Python package.
<https://github.com/neo4j/neo4j-graphrag-python>
- [4] Neo4j: “What is GraphRAG?”
<https://neo4j.com/blog/genai/what-is-graphrag/>
- [5] LangChain documentation: Retrieval. <https://docs.langchain.com/oss/python/langchain/retrieval>
- [6] LlamaIndex documentation: Introduction to RAG. <https://developers.llamaindex.ai/python/framework/understanding/rag/>
- [7] Yang et al. *HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering*. EMNLP 2018.
- [8] Ho et al. *Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps*. COLING 2020 (2WikiMultiHopQA).
- [9] Honnibal & Montani. *spaCy: Industrial-strength Natural Language Processing in Python*.
- [10] Robertson & Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. FNTIR 2009.

Roles and Contributions

CECS-551 · Team 8 · TraceGraph

All four members contributed to the proposal, the feasibility study, and the prototype. Below we record per-author primary responsibilities and headline deliverables. Pair-programming and review were continuous; the assignments listed are the *primary* owners of each area, not the only contributors.

Member	Primary Role	Headline Contributions
Utkarsh Balu Lubal	Retrieval & traversal lead	Designed and implemented the seed retriever (BM25, entity, embedding fusion) and the bounded-traversal module (<code>retrieval/retriever.py</code> , <code>retrieval/traversal.py</code>); tuned score-fusion weights; wrote the score-fusion section of the report.
Kashif Manzer	Agent loop & web console lead	Implemented the revisitable callback agent (<code>agent/callbacks.py</code> , <code>agent/tracegraph_agent.py</code>); built the Next.js web console and the <code>/api/graph</code> and <code>/api/answer</code> bridge to the Python CLI; produced the demo screen-recording.
Mayank Maurya	Graph construction & evaluation lead	Authored the graph builder (<code>graph/builder.py</code>) and the edge-creation logic for NEXT, SHARED_ENTITY, LEXICAL_SIM, and RELATION; implemented the metrics module and the ablation runner; produced Table 3.
Dipak Yadav	Data pipeline & docs lead	Built the data-preparation layer (chunking, preprocessing, metadata enrichment); wrote the prepared-dataset CLIs for the demo corpus and HotpotQA; led the technical documentation and assembled this final report.

Shared Responsibilities

Beyond primary ownership, the team shared: weekly design reviews; pair-programming on the agent–retrieval interface; peer review of every pull request before merge; joint authorship of the four phase reports (Phase 1 proposal, Phase 2 feasibility, Phase 3 prototype, technical documentation); and rehearsals and recording of the final demo. Source code, configs, and demo data are available in the public repository accompanying this report.